

HarnessFlow and MiHaCoBench: A Portable Multi-Agent Workflow Pack for AI Coding Assistants and a Harness-Level Benchmark

Hang Yu¹

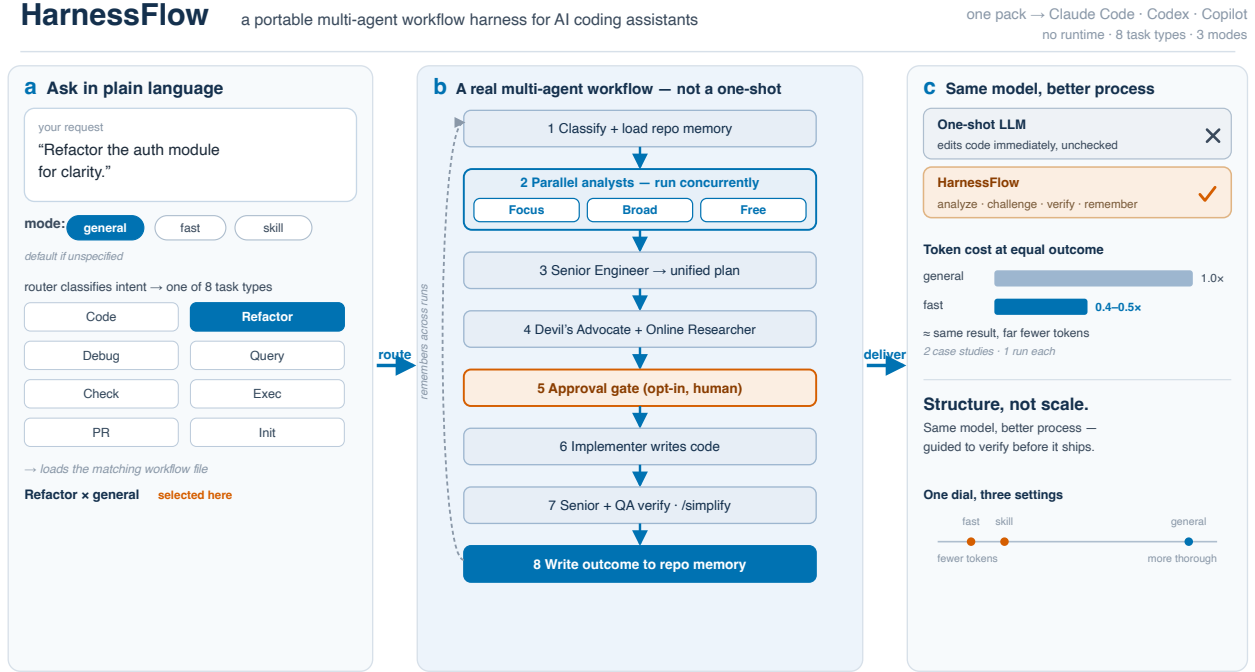


Figure 1. *HarnessFlow at a glance.* (a) A plain-language request, with an optional mode marker, is classified into one of eight task types and routed to the matching workflow. (b) That workflow runs as a multi-agent pipeline—parallel analysts, a synthesized plan, a Devil’s Advocate and an Online Researcher, an opt-in human gate, implementation, a Senior + QA verification pass, and a write-back to persistent repository memory. (c) The intuition: for a fixed model, structure rather than scale yields a checked result, and the three modes trade thoroughness against token cost. Token figures are from two case studies (§4), a single run per arm—preliminary, not a powered benchmark.

Abstract

Modern AI coding assistants are powerful but typically *one-shot*: given a request they edit code immediately, with no structured analysis, no self-criticism, no validation, and no memory across requests. We present **HarnessFlow**, a portable Markdown instruction pack—no runtime,

no build step—that turns a compatible assistant (Claude Code, Codex, VS Code + Copilot) into a disciplined multi-agent system (evaluated here on Claude Code only): by design it classifies each request into one of eight categories, fans out parallel analysis agents, challenges its own plan, validates with a QA pass, and records what it learned to persistent repository memory. The structure is motivated by evidence that how an LLM’s work is organized—chain-of-thought, self-critique, multi-agent debate, and role specialization—changes its outputs for a fixed model (Wei et al., 2022; Madaan et al., 2023; Du et al., 2024; Hong et al.,

¹HarnessFlow Project. Correspondence to: Hang Yu <hangyu8123@gmail.com>.

2024). HarnessFlow ships every workflow in three modes—GENERAL (thorough), FAST (token-efficient), and SKILL (community-skill-backed). In two case studies, with the model held fixed, the FAST mode reached the same successful outcome as GENERAL while spending only 39–50% of the tokens; we report these as preliminary, single-instance demonstrations, not statistically validated results. To make such effects measurable we also built **MiHaCoBench**, a Python benchmark that scores the *harness* rather than the model: it holds the model fixed across arms, enforces grader integrity as its own correctness test, isolates each task to its specification, and—by design—targets a failure frontier where pipelines may diverge.

1. Why a Harness?

The capability of large language models on code has advanced faster than the *process* wrapped around them. Left to their defaults, today’s coding assistants behave like an over-eager junior engineer: they read the prompt and start editing, skipping the steps a careful practitioner takes—surveying the codebase, weighing alternatives, stress-testing assumptions, verifying the result, and remembering context for next time. A growing body of work indicates the model is often not the binding constraint: structured reasoning (Wei et al., 2022; Wang et al., 2023), reasoning interleaved with actions (Yao et al., 2023), iterative self-critique (Madaan et al., 2023; Shinn et al., 2023), and role-specialized multi-agent collaboration (Hong et al., 2024; Qian et al., 2024; Wu et al., 2023) all change outcomes for a *fixed* model, and on real software tasks the surrounding agent scaffold can substantially affect outcomes (Yang et al., 2024).

HarnessFlow packages this discipline as a *drop-in instruction pack*: no runtime, no `npm install`, no build step—only Markdown. Copied into a repository, it gives a compatible assistant a structured workflow for *every* request (Figure 1): *classify* the prompt, *route* it to a real multi-step workflow, *analyze* the code from several angles in parallel, *challenge* the plan, *validate* the implementation, and *record* the outcome to repository memory so later requests start with real context. The pack is designed to run unmodified on three host tools (Claude Code, Codex, and Copilot); the evaluation in this report was conducted on Claude Code only.

Contributions.

- **A portable, runtime-free coding harness.** Unlike standalone multi-agent frameworks (MetaGPT, ChatDev, AutoGen) and bespoke agent scaffolds (SWE-agent), HarnessFlow ships *no runtime*—an open Mark-

down pack that routes any request into a multi-agent workflow (classify → analyze → challenge → gate → implement → verify → remember), interpreted unmodified by three host tools (§2).

- **Three cost–thoroughness modes.** GENERAL, FAST, and SKILL variants of every workflow, exposing an explicit dial instead of one fixed pipeline (§3).
- **A harness-level benchmark.** MiHaCoBench scores the *pipeline*, not the model, with grader-integrity self-checks, spec-only isolation, and failure-frontier tasks (§5).
- **Preliminary evidence.** Two case studies in which FAST matched GENERAL’s outcome at $\sim 2\times$ lower token cost, reported with their $n=1-2$ limitations (§4).
- **A testable hypothesis.** From exploratory runs we hypothesize that *well-specified* tasks under-discriminate harnesses, so separating them requires failure-frontier tasks (§5).

2. HarnessFlow Architecture

HarnessFlow is five cooperating pieces of Markdown. (1) A per-platform *router* (CLAUDE.md, AGENTS.md, or copilot-instructions.md) classifies each prompt. (2) Three *workflow families* under `workflow/` hold eight category instruction files each. (3) Fifteen *worker sub-agents* under `agents/` provide the roles the workflows orchestrate. (4) A shared *workflow contract* fixes cross-cutting rules. (5) A *philosophy* document encodes the behavioral core. Persistent *repository memory* lives under `repo-info/`.

Routing. The router reads the prompt and selects exactly one of eight categories—*Code*, *Refactor*, *Debug*, *Query*, *Correctness Check*, *Exec*, *PR Creation*, and *Initialize*. Trigger keywords are soft signals; classification is by primary intent. An explicit `mode:` marker (or its absence) selects one of three families. The router then reads the matching `<category>.instructions.md` file in full and follows it step-by-step; multiple intents are handled sequentially.

The pipeline. Figure 1(b) shows the GENERAL *Code* workflow, representative of the mutating workflows. The main agent condenses repository memory into a context digest, then launches three analysts in parallel—*Focus* (only the most relevant files), *Broad* (the whole pipeline upstream to downstream), and *Free* (its own judgment). A Senior Engineer reconciles their plans; the main agent synthesizes a final plan; a *Devil’s Advocate* attacks it for regressions while an *Online Researcher* checks live sources. After an opt-in approval gate, an Implementer writes the code, and a parallel Senior + QA pass verifies it before the outcome is written back to memory. The *Refactor* workflow widens

the fan-out to six specialist analysts reviewed by a Principal Engineer.

Why these roles. The separate *Devil’s Advocate* and *QA* stages are deliberate: iterative self-critique and multi-agent debate can improve an LLM’s outputs (Madaan et al., 2023; Shinn et al., 2023; Du et al., 2024), but intrinsic self-correction *without* an external signal is unreliable and can even degrade quality (Huang et al., 2024; Olausson et al., 2024). HarnessFlow therefore pairs critique with executable checks—a QA pass that can run the code—following the execution-grounded self-debugging recipe (Chen et al., 2023).

Cross-cutting guarantees. The contract enforces *model parity* (subagents must not downgrade from the orchestrator’s model, to avoid silently losing quality in a sub-call), a *two-mode approval gate* (autonomous by default; opt-in plan-only review triggered by phrases like “plan only” or “no file changes”), and universal safety rules (never commit to GitHub, never write spam files, never use `sudo`). The philosophy document embeds four guidelines—*Think Before Coding*, *Simplicity First*, *Surgical Changes*, and *Goal-Driven Execution*—so the workflow encourages every changed line to trace back to the request.

3. Three Workflows, One Pack

Every category ships in three modes that share the *same* skeleton—read the contract, gather context, plan, challenge, gate, implement, verify, record—but differ in how much machinery they spend (Table 1).

GENERAL (mode: `general`, the default). The thorough variant with the widest fan-out: three to six dedicated planning analysts, a Senior or Principal Engineer plan review, a Devil’s Advocate / Online Researcher critique pair, and a *separate* Senior + QA verification stage that can run the full pipeline end-to-end. Use it when correctness and breadth dominate cost.

FAST (mode: `fast`). The token-efficient twin. It folds the planning analysts and the post-hoc review subagents *into the main agent*, which does analysis, planning, and implementation inline and spawns subagents in exactly one step—the Devil’s Advocate + Online Researcher challenge. Verification becomes a structured self-review (“assume every change is wrong and explain why”) plus native review skills and a single remediation pass. The instruction files are roughly half the size of their GENERAL counterparts, and in our two case studies the resulting outcome was unchanged (§4).

SKILL (mode: `skill`). The FAST pipeline with eligible steps—planning, implementation, debugging, challenge, research, correctness—delegated

Table 1. The three workflow modes. “Subagent spawns” counts the dedicated parallel roles a *Code* run launches.

	GENERAL	FAST	SKILL
Planning analysts	3–6	inline	inline [†]
Subagent spawns (Code)	~8	2	2 [†]
Review stage	Senior+QA	self/native	skill/native
Approval gate	opt-in	opt-in	opt-in
Repo memory	yes	yes	yes
Relative token cost [‡]	1.0×	0.4–0.5×	≈ fast [§]

[†] SKILL mirrors FAST’s structure, routing eligible steps to community skills with inline fallbacks. [‡] Observed in two case studies (§4); illustrative, not a calibrated average. [§] Expected by design; not measured.

to confirmed community skills, each verified at ≥ 1000 GitHub stars (checked 2026-06-16; e.g. `writing-plans` and `systematic-debugging` from the `obra/superpowers` collection, ~230k stars) and kept behind an *inline fallback* so the workflow never blocks if a skill is absent. A team rides battle-tested external tooling for generic steps while keeping HarnessFlow’s structure, gates, and memory intact.

4. What Users Get

Scope of evidence. All quantitative comparisons below come from one greenfield build (“ShapeLab”) and one SWE-bench task (`sympy__sympy-24213`), at a fixed model, with a single run per arm. They are preliminary demonstrations meant to illustrate the cost–thoroughness trade-off, not statistically validated results.

The intended benefit is that *thoroughness becomes a dial, not a gamble*. We examined the trade-off in these two case studies with the *model held constant* (Sonnet-class subagents, Opus-class orchestrator) so the comparison isolates the harness, not the model: a single ~1,000-line greenfield OpenCV + scikit-learn build, and a single real SWE-bench bug fix. In both cases, FAST reached the same successful outcome as GENERAL while spending only 39–50% of the tokens (Figure 2), and in the single ShapeLab build it produced the leanest fully-documented code among the three arms tested (Table 2). With only two tasks, these are illustrative rather than conclusive.

Beyond cost, the pack is intended to pay off in three ways. *Portability*: one instruction pack is designed to standardize review discipline across Claude Code, Codex, and Copilot (evaluated here on Claude Code only). *Quality under self-criticism*: in the single SWE-bench task the no-harness baseline produced a non-canonical, unverified patch while both harnessed modes produced the canonical fix; in the single ShapeLab build the baseline passed 7/10 contract

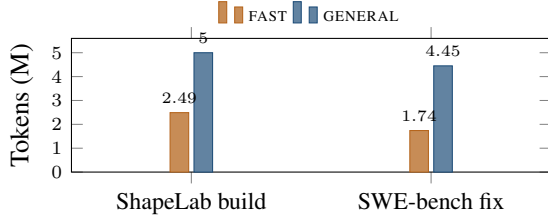


Figure 2. Token cost in two case studies where FAST reached the same resolved/verified result as GENERAL, at 2.0–2.6 $\times$ lower cost. Single run per arm; no variance is reported. The no-harness baseline is omitted—it is cheaper still but does not reach the verified outcome (Table 2).

Table 2. Quality metrics from the single ShapeLab build (the last row is the single SWE-bench fix). The baseline is the same model with no harness; one run per arm, no variance.

Metric ($n=1$)	baseline	FAST	GENERAL
Code lines (AST)	1,225	860	1,173
Docstring coverage	96.3%	100%	100%
Contract tests passed	7/10	10/10	10/10
Self-verified pre-ship	none	32 green	yes
SWE canonical fix	$\times$	$\checkmark$	$\checkmark$

tests versus 10/10 for the harnessed modes—consistent with, though far smaller than, the literature on execution-grounded checking (Chen et al., 2023). *Memory and safety*: outcomes persist to `repo_info/` so the next request starts informed, while opt-in plan review and bans on commits and `sudo` keep autonomy bounded.

5. MiHaCoBench: Scoring the Harness

Building HarnessFlow surfaced a measurement problem: standard code benchmarks (Chen et al., 2021; Austin et al., 2021; Jain et al., 2024) score a *model* in isolation, but a harness contributes prompt handling, multi-step planning, tool use, and long-horizon consistency that those benchmarks never exercise. We therefore built **MiHaCoBench**, both the development test bench for HarnessFlow and a standalone instrument intended for evaluating other coding pipelines (so far exercised only on HarnessFlow’s own arms). Its design draws on SWE-bench, BigCodeBench, LiveCodeBench, ClassEval, and DS-1000 (Jimenez et al., 2024; Zhuo et al., 2024; Jain et al., 2024; Du et al., 2023; Lai et al., 2023). Four properties distinguish it.

1. It scores the harness, not the model. Any arm—no-harness baseline, FAST, SKILL, or GENERAL—can be run over the same tasks at the *same* model with verified parity, so that score deltas are *intended* to be attributable to process rather than capability; with single runs, observed deltas may also reflect run-to-run variance.

2. Grader integrity is its own test. Borrowing SWE-bench’s FAIL_TO_PASS / PASS_TO_PASS discipline (Jimenez et al., 2024), each task ships a hidden *gold* reference the grader must accept and a deliberately *broken* reference it must reject. The default `--self-check` mode verifies all 69 graders valid (gold passes, broken fails) *before* any candidate is scored; graders test only the public contract, so a harness cannot be rewarded for matching an internal layout.

3. Spec-only isolation and contamination control. The scaffolder copies *only* each task’s `TASK.md` (plus committed inputs) into an external workspace and refuses any target inside the repository, so an agent never sees graders, references, or sibling tasks. Reproducibility is pinned: Python 3.11.5, committed datasets, fixed seeds, `PYTHONHASHSEED=0`. Contamination is resisted with domain-specific entry-point names, twisted classics, and a dated `TASK.md` per task for temporal filtering, in the spirit of contamination-free evaluation (Jain et al., 2024).

4. A failure-frontier task design. The curated suite is 69 weighted tasks at the time of these runs (the live suite has since grown) across ten populated categories (an eleventh is reserved), weighted from easy=1 to competitive=8 so a harness cannot inflate its score by acing only the easy ones. In exploratory single-run comparisons we observed that well-specified tasks under-discriminate: a careful frontier model reads the spec and one-shots them, and the arms tie. The suite therefore targets the regime where single shots fail—hard complexity gates that physically time out wrong-complexity solutions (a regime where models are known to struggle (Chambon et al., 2025)), multi-file fault localization, statistical surface-form twists where the memorized answer is wrong, and long state cascades. Based on these preliminary observations we *hypothesize* that separating harnesses requires failure-frontier tasks, not merely well-specified ones.

What we observe. Exploratory single-run ($n=1$) comparisons on a 79-task superset, with three arms (no-harness, FAST, SKILL; GENERAL was not run here), partially bear this out: the arms tie on the large majority of tasks (72/79), and the few that discriminate are exactly the failure-frontier items the design targets. Tellingly, the discrimination cuts *against* a naive “more process is better” prior. On `cb08_cursor_paginate`—a boundary/tie-break trap—the lighter FAST pipeline shipped a wrong variant (0.44) while *both* the no-harness baseline and the heavier SKILL arm solved it (1.00); and overall on this single run the no-harness baseline (76/79) edged out FAST (75/79), with only SKILL (78/79) clearly ahead. We report these as directional, not statistically significant; their value is in showing that the *instrument* can localize where harnesses diverge—and even flag harness *regressions*, not only gains.

6. Related Work

Multi-agent coding systems. MetaGPT (Hong et al., 2024), ChatDev (Qian et al., 2024), and AutoGen (Wu et al., 2023) are standalone Python frameworks: each ships its own runtime, agent classes, and orchestration loop, and an adopter installs and runs that system; SWE-agent (Yang et al., 2024) pairs a model with a bespoke agent–computer interface. HarnessFlow occupies a different point in the design space—it carries *no runtime and no code*. It is a portable Markdown instruction pack copied into a repository and interpreted by whichever assistant the team already uses, turning that host into a router-driven, multi-mode, repository-memory-equipped pipeline. The contribution is thus not a new agent runtime but a host-agnostic, version-controllable *specification* of disciplined process. **Reasoning and self-critique.** HarnessFlow composes established techniques into roles—chain-of-thought (Wei et al., 2022; Wang et al., 2023), reason–act (Yao et al., 2023), self-refinement (Madaan et al., 2023; Shinn et al., 2023), and debate (Du et al., 2024)—and, because intrinsic self-correction is unreliable without an external signal (Huang et al., 2024; Olausson et al., 2024), grounds them in executable QA (Chen et al., 2023). **Benchmarks.** Standard suites score a model (Chen et al., 2021; Austin et al., 2021; Jain et al., 2024); MiHaCoBench instead scores the harness (§5).

7. Limitations

Our evidence is preliminary. (i) *Sample size:* all quantitative comparisons rest on two case studies (one greenfield build, one SWE-bench fix) with a single run per arm; we report no variance and run no significance tests, so the token and quality gaps are directional. (ii) *Single host:* although HarnessFlow is designed for Claude Code, Codex, and Copilot, every result here is from Claude Code only—cross-host portability is asserted by design, not demonstrated. (iii) *Benchmark coverage:* MiHaCoBench has so far been exercised only on HarnessFlow’s own arms with $n=1$ runs, and (§5) one of those runs surfaced a *regression*—FAST underperforming the no-harness baseline—so its discriminative power is shown anecdotally, not established statistically. Tables 1 and 2 are illustrative single instances.

8. Conclusion

Our case studies suggest that some of the value left on the table by AI coding assistants may be recoverable with *process* rather than a bigger model. HarnessFlow turns a one-shot tool into a more disciplined collaborator—classify, analyze in parallel, self-challenge, validate, remember—with three modes that price thoroughness explicitly (FAST matched GENERAL at roughly half the cost in two case studies). MiHaCoBench is designed to make such claims testable by

scoring the pipeline, not the model; in preliminary runs it localized failure-frontier tasks where the arms diverged—in one case flagging a *regression* (FAST below the no-harness baseline), exactly the signal such an instrument should expose. Together they form a development loop—build the harness, measure the harness—that we believe is the right granularity for the agentic-coding era. The open questions are scale and rigor: powering the comparisons beyond $n=1-2$, testing portability on Codex and Copilot, and growing the failure-frontier suite.

Impact Statement

HarnessFlow structures how AI coding assistants operate, and MiHaCoBench evaluates such harnesses. The intended impact is more reliable, reviewable, and reproducible AI-assisted software development—adding analysis, self-criticism, and verification to otherwise one-shot tools, and enabling measurement of process rather than model capability. As with any system that raises the autonomy of code-writing agents, there is a risk that added fluency leads practitioners to over-trust generated code; HarnessFlow mitigates this with executable QA checks, an opt-in human approval gate, and safety rules that forbid committing to remotes and using elevated privileges. We do not anticipate societal consequences beyond those generally associated with AI-assisted programming.

Software and Data

HarnessFlow (the instruction pack) and MiHaCoBench (the benchmark, its grader-integrity self-check, and the scaffold) are released as open artifacts. The benchmark pins Python 3.11.5, committed datasets, fixed seeds, and PYTHONHASHSEED=0 for reproducibility.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chambon, P., Roziere, B., Sagot, B., and Synnaeve, G. BigO(Bench) – can LLMs generate code with controlled time and space complexity? *arXiv preprint arXiv:2503.15242*, 2025.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- Du, X., Liu, M., Wang, K., Wang, H., et al. Clas-

- sEval: A manually-crafted benchmark for evaluating LLMs on class-level code generation. *arXiv preprint arXiv:2308.01861*, 2023.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.
- Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jain, N., Han, K., Gu, A., Li, W.-D., et al. LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Lai, Y., Li, C., Wang, Y., Zhang, T., et al. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning (ICML)*, 2023.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Olausson, T. X., Inala, J. P., Wang, C., Gao, J., and Solar-Lezama, A. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., et al. ChatDev: Communicative agents for software development. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., et al. Big-CodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.